

Análisis estadístico aplicado a la calidad del vino

Universidad Nacional Guillermo Brown



Bigorra, Paula Antonella
Pastori Portabales, Santiago

Florencia Statti

Inferencia Estadística y Reconocimiento de Patrones

INTRODUCCIÓN.....	3
MODELO DE REGRESIÓN.....	3
Regresión lineal múltiple.....	3
Implementación del Modelo.....	3
Diagnóstico visual.....	4
Resultado del modelo.....	5
Evaluación del ajuste y validez del modelo.....	5
Regresión Ridge y LASSO.....	7
Implementación del Modelo.....	7
1. Preparación de los datos.....	7
2. Definición de la grilla de penalización.....	7
3. Ajuste de los modelos Ridge y LASSO.....	8
4. Validación cruzada y selección del mejor lambda.....	8
5. Visualización de los coeficientes ajustados.....	9
6. Evaluación del desempeño predictivo.....	9
7. Visualización de trayectorias de coeficientes.....	10
Diagnóstico Visual.....	10
Resultados del Modelo.....	11
Evaluación del ajuste y validez del modelo.....	12
MODELOS DE CLASIFICACIÓN.....	13
Regresión Logística Multinomial.....	13
Implementación del modelo.....	13
1. Ajuste del modelo.....	13
2. Visualización de los Resultados e implementación de métricas.....	14
Diagnóstico Visual.....	15
Resultados del modelo.....	15
Evaluación del ajuste y validez del modelo.....	16
K vecinos más cercanos (KKN).....	17
Implementación del Modelo.....	17
1. Normalización.....	17
2. Candidatos de K.....	17
3. Métricas de clasificación y elección del mejor K.....	18
4. Entrenamiento del modelo Final.....	18
5. Evaluación de KNN.....	19
Diagnóstico Visual.....	19
Resultados del modelo.....	20
Evaluación del ajuste y validez del modelo.....	21
Bayes Ingenuo / QDA / LDA.....	22
Implementación del Modelo.....	22
Diagnóstico Visual.....	25
Resultados del modelo.....	27
Evaluación del ajuste y validez del modelo.....	28
CONCLUSIÓN.....	30

INTRODUCCIÓN

El presente informe tiene como objetivo aplicar técnicas de inferencia estadística y reconocimiento de patrones sobre un conjunto de datos enológicos, con el fin de analizar la calidad del vino tinto. Para ello se utiliza el dataset “*winequality-red.csv*”, el cual contiene un total de 1.599 observaciones y 12 variables numéricas, incluyendo la variable objetivo *quality*. Las variables registran distintas características del vino, como acidez, pH, densidad y contenido de alcohol, entre otras.

A partir del conjunto de datos provisto por la docente, se procede directamente a la implementación de los métodos estadísticos establecidos en la consigna, con el fin de interpretar los resultados obtenidos y evaluar su comportamiento sobre el conjunto analizado.

MODELO DE REGRESIÓN

Regresión lineal múltiple

Implementación del Modelo

Al implementar un modelo de regresión lineal múltiple determinamos como variable respuesta la calidad del vino (*quality*) y consideramos todas las demás variables del conjunto de datos como explicativas. Para ello se emplea la función `lm()` que ajusta el modelo y lo guarda con el nombre de `modelo_lineal`. Luego, con `summary(modelo_lineal)`, se obtiene un resumen estadístico que incluye estimaciones de coeficientes, errores estándar, p-valores y medidas de ajuste. Este análisis permite identificar las variables que son significativas para nuestra variable respuesta.

A continuación, se generaron los gráficos de diagnóstico mediante `plot(modelo_lineal)`, organizados en una cuadrícula de 2x2 con `par(mfrow = c(2, 2))`. Estos gráficos permiten evaluar visualmente si se cumplen los cuatro supuestos clásicos de la regresión lineal: linealidad, normalidad de los residuos, homocedasticidad y ausencia de observaciones influyentes.

Finalmente, se construyó un segundo modelo (`modelo_nuevo`) utilizando únicamente aquellas variables que resultaron estadísticamente significativas en el primer ajuste: *volatile.acidity*, *chlorides*, *pH*, *sulphates* y *alcohol*. Este nuevo modelo busca evaluar si restringir el conjunto de predictores mejora el ajuste general, reduce el error de predicción o modifica la

interpretación de los coeficientes. Al aplicar `summary(modelo_nuevo)`, se obtiene una nueva salida estadística que puede compararse con la del modelo original para determinar si la simplificación del modelo contribuye a una representación más precisa y eficiente

```

5 #=====
6 #REGRESIÓN LINEAL MULTIPLE
7 #=====
8
9 "Creamos el modelo con quality como la variable respuesta y los demás datos como las variables explicativas
10 para luego mostrar un resumen estadístico"
11
12 modelo_lineal <- lm(vinos$quality~.,data=vinos)
13 summary(modelo_lineal)
14
15 "Creacion de graficos para evaluar las cuatro supuestos clásicos de la regresión lineal"
16
17 par(mfrow = c(2, 2))
18 plot(modelo_lineal)
19
20 "Nuevo Modelo con las variables estadísticamente significativas"
21
22 modelo_nuevo <- lm(quality~volatile.acidity+chlorides+density+pH+sulphates+alcohol, data=vinos)
23 summary(modelo_nuevo)

```

Diagnóstico visual

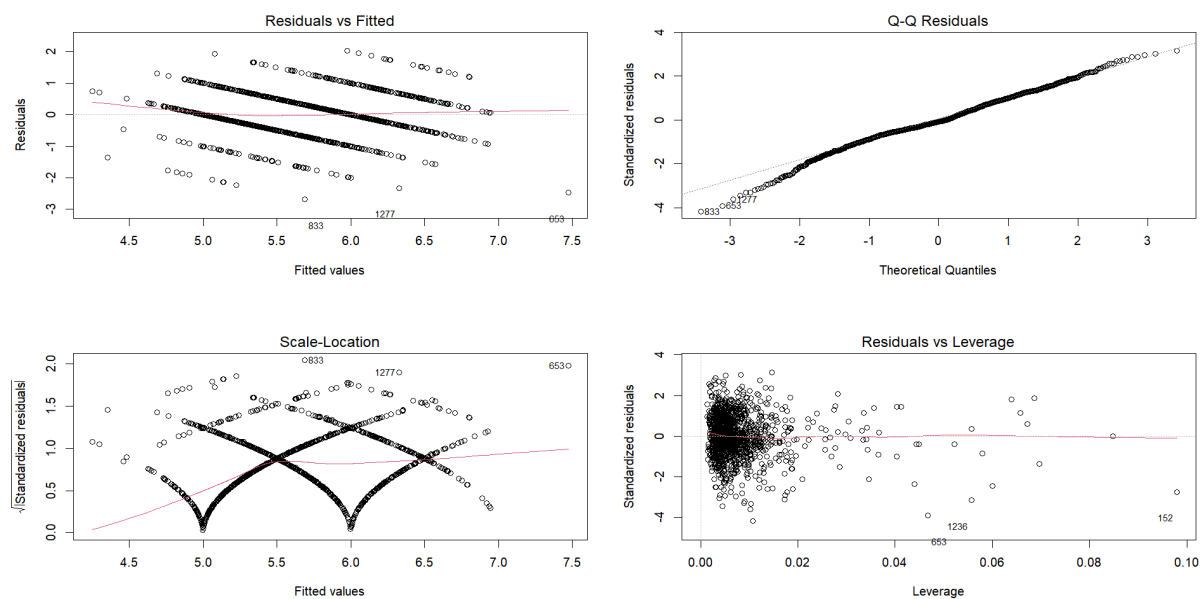


Figura 1: Conjunto de gráficos (Residuals vs Fitted, Normal Q-Q, Scale-Location, Residuals vs Leverage) que evalúan la validez del modelo de regresión lineal en el dataset presentado.

Resultado del modelo

```
Call:
lm(formula = vinos$quality ~ ., data = vinos)

Residuals:
    Min       1Q   Median       3Q      Max
-2.68911 -0.36652 -0.04699  0.45202  2.02498

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   2.197e+01  2.119e+01   1.036   0.3002
fixed.acidity   2.499e-02  2.595e-02   0.963   0.3357
volatile.acidity -1.084e+00  1.211e-01  -8.948 < 2e-16 ***
citric.acid    -1.826e-01  1.472e-01  -1.240   0.2150
residual.sugar  1.633e-02  1.500e-02   1.089   0.2765
chlorides      -1.874e+00  4.193e-01  -4.470 8.37e-06 ***
free.sulfur.dioxide 4.361e-03  2.171e-03   2.009 0.0447 *
total.sulfur.dioxide -3.265e-03  7.287e-04  -4.480 8.00e-06 ***
density        -1.788e+01  2.163e+01  -0.827   0.4086
pH             -4.137e-01  1.916e-01  -2.159   0.0310 *
sulphates       9.163e-01  1.143e-01   8.014 2.13e-15 ***
alcohol        2.762e-01  2.648e-02  10.429 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.648 on 1587 degrees of freedom
Multiple R-squared:  0.3606,    Adjusted R-squared:  0.3561
F-statistic: 81.35 on 11 and 1587 DF,  p-value: < 2.2e-16
```

Figura 2: Devolución de Summary(modelo_lineal)

```
Call:
lm(formula = quality ~ volatile.acidity + chlorides + density +
  pH + sulphates + alcohol, data = vinos)

Residuals:
    Min       1Q   Median       3Q      Max
-2.60091 -0.36252 -0.06116  0.46943  1.97588

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   4.52129   10.70843   0.422 0.672923
volatile.acidity -1.06855   0.10094 -10.586 < 2e-16 ***
chlorides      -1.92931   0.40063  -4.816 1.61e-06 ***
density        -0.50603   10.56117  -0.048 0.961790
pH             -0.41868   0.12037  -3.478 0.000518 ***
sulphates       0.84834   0.11210   7.568 6.38e-14 ***
alcohol        0.30508   0.01876  16.259 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.6532 on 1592 degrees of freedom
Multiple R-squared:  0.3483,    Adjusted R-squared:  0.3459
F-statistic: 141.8 on 6 and 1592 DF,  p-value: < 2.2e-16
```

Figura 3: Devolución de Summary(modelo_nuevo)

Evaluación del ajuste y validez del modelo

El resumen estadístico del modelo (summary(modelo_lineal)) muestra que algunas variables como volatile.acidity, chlorides, pH, sulphates y alcohol presentan p-valores muy bajos, lo que indica que tienen una influencia significativa sobre la calidad del vino. En cambio, otras variables como citric.acid, residual.sugar, free.sulfur.dioxide y total.sulfur.dioxide no resultan significativas, lo que sugiere que su aporte al modelo es débil o nulo.

El error estándar residual es de 0.648, lo que significa que, en promedio, las predicciones del modelo se desvían 0.648 unidades respecto a los valores reales de calidad. Este valor representa la dispersión de los residuos alrededor de la línea de regresión ajustada. Aunque puede parecer bajo en términos absolutos, su interpretación depende del rango de la variable respuesta (quality). En este caso, considerando que la calidad del vino se mide en una escala acotada, una desviación promedio de 0.648 indica que el modelo tiene un margen de error considerable. Además, el R^2 ajustado es de apenas 0.3561, lo que significa que el modelo explica solo el 35% de la variabilidad en la calidad del vino. Esto sugiere que hay relaciones no capturadas por el modelo lineal, y que su capacidad predictiva es limitada.

Asimismo, implementamos un nuevo modelo lineal únicamente con las variables estadísticamente significativas obtenidas anteriormente para una comparación. Verificando si los valores devueltos por R^2 disminuyen o crecen en relación al modelo anterior.

A pesar de las nuevas implementaciones hechas, notamos que el R^2 disminuyó insignificamente en relación al modelo anterior. En base a esta nueva observación podríamos concluir que sin importar la combinación de variables que tomemos no influirá en

la mejora de la representación de la variable respuesta.

Los gráficos de diagnóstico generados por `plot(modelo_lineal)` permiten evaluar si se cumplen los supuestos clásicos de la regresión lineal, aportando evidencia visual sobre el comportamiento y la validez del ajuste.

El Gráfico Residuals(Eje X) vs Fitted (Eje Y) permite evaluar si se cumple el supuesto de linealidad en el modelo. Para que esto se efectúe los datos deben estar dispersos aleatoriamente alrededor de la línea horizontal de cero, sin un patrón o tendencia. Además, se debe presentar una recta roja paralela al eje X que interseque al eje Y en el valor 0.

En el modelo, observamos que los puntos siguen un patrón y la línea roja posee una curvatura concluyendo que no cumple con la linealidad, no logrando capturar completamente la relación entre las variables.

En el Gráfico Q-Q Residuals (Eje X= Cuantiles teóricos de una distribución normal, Y= Residuos estandarizados) buscamos observar si los residuos tienden a ser normales, y para que esto se cumpla la totalidad de los puntos deben superponerse a la diagonal. Observamos en el gráfico de la Figura 1 que la línea de puntos sigue la recta en su mayoría, aunque hay desvíos en las colas, lo que sugiere que los residuos son aproximadamente normales.

El Gráfico Scale-Location con X=Valores ajustados e Y= Raíz cuadrada de residuos estandarizados evalúa si se cumple el supuesto de homocedasticidad, para comprobar esto la línea de tendencia en el gráfico debe ser horizontal y los residuos estar dispersos aleatoriamente sin mostrar un patrón creciente o decreciente. Al analizar el gráfico, se puede observar un patrón con dichas características. Además, la línea de tendencia (roja) no se mantiene recta, sino que presenta curvaturas visibles indicando que no se cumple el supuesto de homocedasticidad.

Finalmente, el Gráfico Residuals(Eje Y) vs leverage(Eje X) permite detectar cuánta influencia tiene cada punto sobre el ajuste del modelo. Para que se cumpla el supuesto, los datos deben estar dispersos sin formar patrones, ubicados dentro de los límites e idealmente no deberían aparecer puntos aislados en esquinas o extremos. Observando el gráfico podemos ver que los puntos están mayormente agrupados y no se observan valores extremos, lo que indica que no hay observaciones altamente influyentes en el modelo.

Regresión Ridge y LASSO

Implementación del Modelo

1. Preparación de los datos

```
25 # =====  
26 # Regresión Ridge y Regresión LASSO  
27 # =====  
28  
29 "Creación de la matriz (X = variables predictoras/explicativas ; Y = variable respuesta)"  
30 x <- as.matrix(vinos[, -which(names(vinos) == "quality")])  
31 y <- vinos$quality  
32  
33  
34 "Dividimos el conjunto de datos en train (80%) y test (20%) tomando de manera aleatoria"  
35  
36 set.seed(123)  
37 train <- sample(1:nrow(vinos), 0.8 * nrow(vinos))  
38 test <- setdiff(1:nrow(vinos), train)  
39  
40 "Extrae las filas de train/test de la matriz X y de los valores de quality"  
41 x_train <- x[train, ]  
42 y_train <- y[train]  
43 x_test <- x[test, ]  
44 y_test <- y[test]
```

En el proceso de preparación para aplicar regresión penalizada sobre el dataset de vinos, se comienza por definir la estructura de datos que utilizarán los modelos Ridge y LASSO. Primero, se construye la matriz `x`, que contiene todas las variables predictoras/explicativas del conjunto de datos, excluyendo la columna `quality`, la cual se define como la variable respuesta y se almacena en el vector `y`. Esta conversión a formato matricial es necesaria porque las funciones de ajuste utilizadas en `glmnet` requieren que las variables explicativas estén representadas como matriz numérica.

A continuación, se realiza una partición aleatoria del conjunto de datos, asignando el 80% de las observaciones al conjunto de entrenamiento y el 20% restante al conjunto de test. Esta división permite evaluar el rendimiento del modelo sobre datos no utilizados durante el ajuste, lo que mejora la validez de las conclusiones.

Luego, se extraen las filas correspondientes a cada subconjunto tanto para la matriz `x` como para el vector `y`. Así se generan cuatro objetos: `x_train` e `y_train`, que contienen las variables predictoras y la variable respuesta del conjunto de entrenamiento, y `x_test` e `y_test`, que contienen los mismos elementos pero del conjunto de test. Estos objetos serán utilizados para entrenar los modelos Ridge y LASSO y evaluar su capacidad predictiva sobre los datos.

2. Definición de la grilla de penalización

```
47 "Creacion de una grilla para obtener el conjuntos de valores candidatos para lambda"  
48  
49 grilla <- 10^seq(10, -2, length = 100)
```

En esta etapa del análisis se implementan los modelos de regresión penalizada Ridge y LASSO, con el objetivo de penalizar cuánto pueden crecer los coeficientes del modelo. El parámetro lambda es quien controla ese límite para evitar el sobreajuste: si lambda es muy chico, el modelo se comporta de forma similar a una regresión lineal clásica, mientras que si lambda es muy grande, especialmente en el caso de LASSO, algunos coeficientes se reducen a cero, haciendo que el modelo ignore esas variables.

Para ello, se crea primero una grilla de 100 valores candidatos para lambda, distribuidos entre 10^{10} y 10^{-2} . Esta grilla permite explorar una amplia gama de penalizaciones, desde muy fuertes hasta casi nulas.

3. Ajuste de los modelos Ridge y LASSO

```
51 "Uso de los modelos Ridge y LASSO a partir de la grilla para evaluar el efecto de penalización sobre los coeficientes"
52
53 ridge.mod <- glmnet(x[train,], y[train], alpha = 0, lambda = grilla)
54 lasso.mod <- glmnet(x[train,], y[train], alpha = 1, lambda = grilla) #Elimina variables irrelevantes (coeficientes se vuelven cero)
```

Luego, se ajustan los modelos utilizando la función `glmnet`, que permite especificar el tipo de penalización mediante el parámetro alpha: se usa $\alpha = 0$ para Ridge y $\alpha = 1$ para LASSO. Ambos modelos se aplican sobre el conjunto de entrenamiento previamente definido, y se evalúa cómo varían los coeficientes de las variables explicativas en función de los distintos valores de lambda.

4. Validación cruzada y selección del mejor lambda

```
56 "Realizamos validación cruzada en los distintos modelos.Obteniendo el lambda que minimiza el error de validacion"
57
58 set.seed(123)
59 cv.ridge <- cv.glmnet(x[train,], y[train], alpha = 0, lambda = grilla)
60 cv.lasso <- cv.glmnet(x[train,], y[train], alpha = 1, lambda = grilla)
61
62 "Guarda los dos mejores lambda que minimiza el error en los modelos"
63 best.lambda.ridge <- cv.ridge$lambda.min
64 best.lambda.lasso <- cv.lasso$lambda.min
65
66 "Visualización de los valores"
67 best.lambda.ridge
68 best.lambda.lasso
```

A continuación, se realiza validación cruzada sobre cada modelo utilizando `cv.glmnet`, quien tiene como objetivo encontrar el valor de lambda que logra el menor error de validación promedio, es decir, el que mejor equilibra ajuste y generalización. Este procedimiento garantiza que el modelo seleccionado tenga buen rendimiento predictivo y no esté sobre

ajustado.

Una vez identificados los valores óptimos de penalización, son almacenados en `best.lambda.ridge` y `best.lambda.lasso`, respectivamente pudiendo visualizar los coeficientes ajustados por cada modelo en el valor óptimo de lambda, lo que permite observar qué variables mantienen influencia en el modelo y cuáles son descartadas. Esta comparación es clave para interpretar la estabilidad del modelo, la relevancia de cada predictor y el efecto de la regularización sobre la estructura del ajuste.

5. Visualización de los coeficientes ajustados

```
70 "Observación los coeficientes finales en el lambda óptimo y cuanto influye cada variable ajustada por los modelos"
71
72 coef(cv.ridge, s = "lambda.min")
73 coef(cv.lasso, s = "lambda.min")
```

El código hace referencia a los modelos `cv.ridge` y `cv.lasso`, que fueron previamente ajustados con validación cruzada. En este contexto los coeficientes reflejan cuánto influye cada variable explicativa en la predicción de la calidad del vino, bajo penalización, es decir, cuanto son ajustadas por los modelos para que el resultado sea preciso y no perjudique.

En Ridge, los coeficientes se reducen pero no se eliminan, mientras que en LASSO algunos se vuelven exactamente cero, lo que implica que el modelo descarta esas variables por completo. Esta diferencia es clave para interpretar la estructura del modelo y su selección de variables.

6. Evaluación del desempeño predictivo

```
77 "Evaluación de la eficiencia de predicción de los modelos Ridge y LASSO usando el conjunto de pruebas"
78
79 pred.ridge <- predict(cv.ridge, s = best.lambda.ridge, newx = x[test,])
80 pred.lasso <- predict(cv.lasso, s = best.lambda.lasso, newx = x[test,])
81
82 "Comparación de las predicciones con los valores reales y calculo de MSE"
83 mean((y[test] - pred.ridge)^2) # MSE RIDGE
84 mean((y[test] - pred.lasso)^2) # MSE LASSO
```

En esta parte, se evalúa la capacidad predictiva de los modelos Ridge y LASSO sobre el conjunto de prueba. Se generan las predicciones (`pred.ridge` y `pred.lasso`) utilizando los modelos ajustados y el valor óptimo de lambda obtenido en la validación cruzada. A continuación, se calcula el error cuadrático medio (MSE) comparando dichas predicciones con los valores reales de la calidad del vino. Esta métrica permite cuantificar la precisión del modelo: cuanto menor es el MSE, mejor es el ajuste logrado. De este modo, se puede

comparar objetivamente el rendimiento de ambos enfoques sobre datos no utilizados durante el entrenamiento.

7. Visualización de trayectorias de coeficientes

```
86 # GRÁFICOS DE TRAYECTORIAS DE COEFICIENTES
87 "Muestra cómo cambian los coeficientes de cada variable cuando varía el valor de lambda. El eje X representa lambda y el eje Y los coeficientes."
88
89 par(mfrow = c(1, 1))
90 plot(ridge.mod, xvar = "lambda", label = TRUE, main = "Trayectoria de coeficientes - Ridge")
91
92 plot(lasso.mod, xvar = "lambda", label = TRUE, main = "Trayectoria de coeficientes - Lasso")
```

Estos gráficos muestran cómo cambian los coeficientes de cada variable explicativa a medida que varía el valor de lambda. En el eje X se representa el nivel de penalización (lambda) y en el eje Y la magnitud de los coeficientes.

Estas trayectorias permiten visualizar el efecto de la regularización sobre cada predictor y entender cómo el modelo simplifica su estructura al aumentar la penalización.

Diagnóstico Visual

Ridge

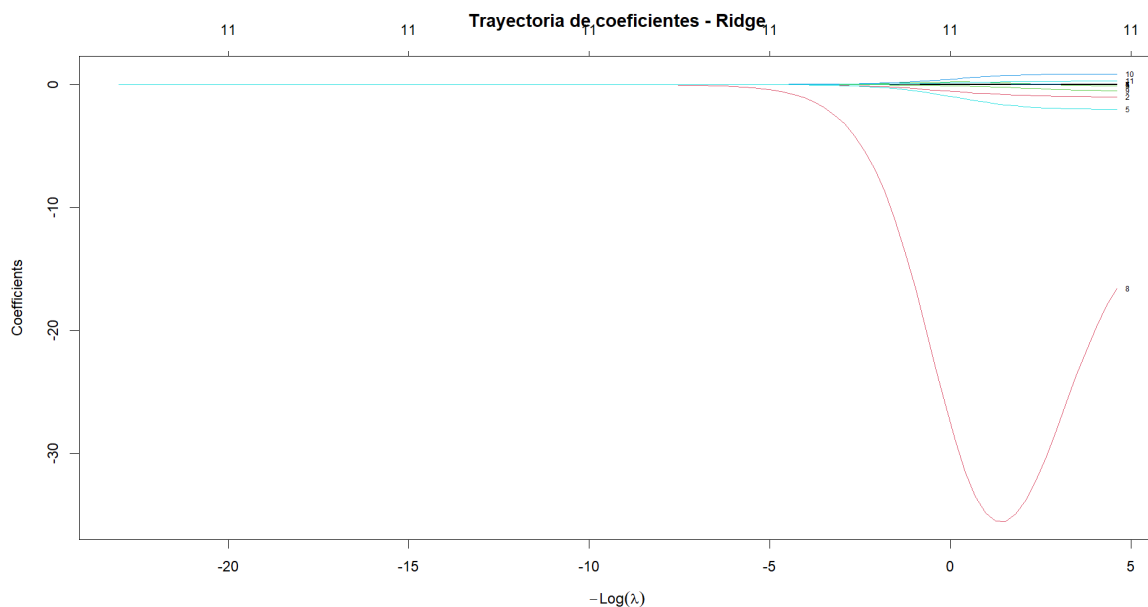


Figura 4: Gráfico de la variación de los coeficientes según lambda, en Ridge

LASSO

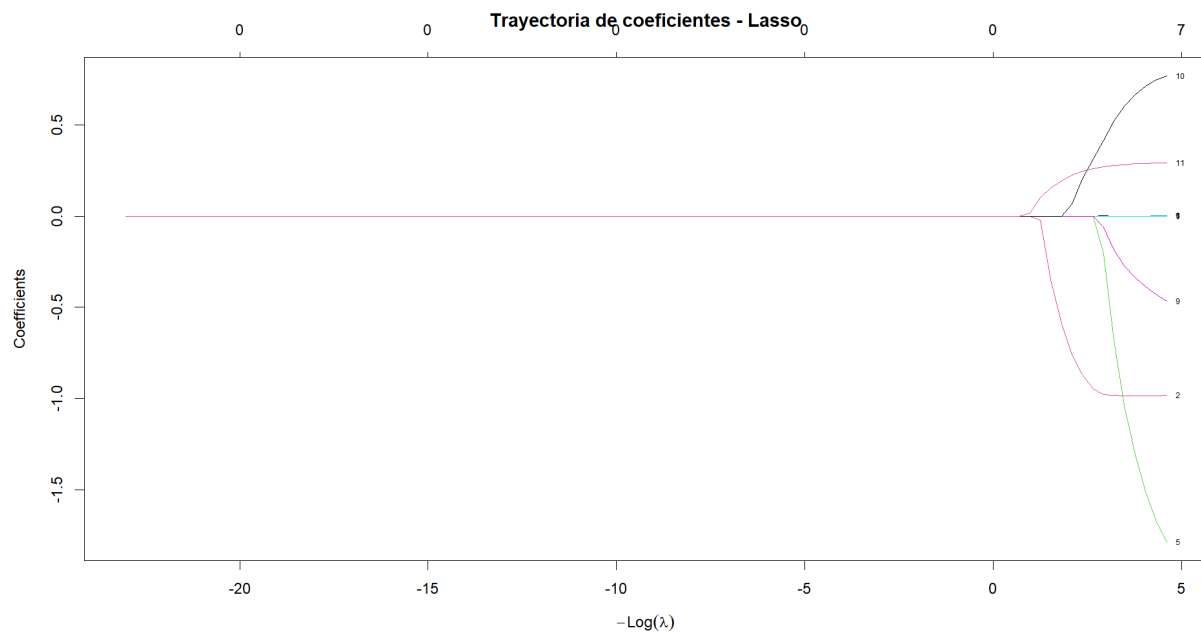


Figura 5: Gráfico de la variación de los coeficientes según lambda, en LASSO

Resultados del Modelo

```
> coef(cv.lasso, s = "lambda.min")
12 x 1 sparse Matrix of class "dgCMatrix"
      lambda.min
(Intercept)      4.415404568
fixed.acidity      .
volatile.acidity  -0.979815920
citric.acid        .
residual.sugar     .
chlorides          -1.783044654
free.sulfur.dioxide 0.001428936
total.sulfur.dioxide -0.002733890
density            .
pH                 -0.467362579
sulphates          0.767926317
alcohol            0.292352705
```

```
> coef(cv.ridge, s = "lambda.min")
12 x 1 sparse Matrix of class "dgCMatrix"
      lambda.min
(Intercept)      30.09397179
fixed.acidity      0.02573640
volatile.acidity  -0.96184759
citric.acid        -0.04599746
residual.sugar     0.01136516
chlorides          -1.95758085
free.sulfur.dioxide 0.00337203
total.sulfur.dioxide -0.00331340
density            -25.87553672
pH                 -0.42446219
sulphates          0.84516055
alcohol            0.26275460
```

Figura 6: Devolución de los coeficientes ajustados

```
> best.lambda.ridge
```

```
[1] 0.04037017
```

```
> best.lambda.lasso
```

```
[1] 0.01
```

```
> mean((y[test] - pred.ridge)^2) # MSE RIDGE
```

```
[1] 0.3425748
```

```
> mean((y[test] - pred.lasso)^2) # MSE LASSO
```

```
[1] 0.3442939
```

Evaluación del ajuste y validez del modelo

Al implementar los modelos de regresión penalizada Ridge y LASSO sobre el dataset de vinos tintos, se observa que ambas técnicas resultan eficaces para predecir la calidad del vino a partir de variables predictoras. La validación cruzada permitió identificar los valores óptimos de penalización: $\lambda = 0.0404$ para Ridge y $\lambda = 0.01$ para LASSO. Estos valores son los que minimizan el error de validación, y al aplicarlos se obtienen los coeficientes finales que definen el impacto de cada variable en la predicción.

Al evaluar el desempeño sobre el conjunto de prueba, se obtuvieron errores cuadráticos medios (MSE) de 0.3426 para Ridge y 0.3443 para LASSO. Dado que la variable respuesta quality tiene un rango acotado entre 3 y 8, estos valores indican que el margen de error promedio es menor a medio punto de calidad, lo cual demuestra que los modelos ajustan correctamente los datos y ofrecen predicciones confiables.

El paquete glmnet normaliza automáticamente las variables predictoras, lo que asegura que la penalización se aplique de forma equitativa entre ellas, sin que las diferencias de escala distorsionen el ajuste. Además, la relación entre las variables explicativas y la respuesta es continua y numérica, lo que valida el uso de regresión penalizada como técnica estadística apropiada.

En cuanto a la estructura del modelo, Ridge conserva todas las variables ajustando sus coeficientes para evitar sobreajuste. Que los coeficientes sean distintos de cero indica que cada variable aporta información al modelo, aunque en distinta magnitud. Por otro lado, LASSO realiza una selección automática, eliminando aquellas variables con menor influencia (coeficientes = 0), lo que simplifica el modelo sin perder capacidad predictiva.

Los gráficos de trayectoria de coeficientes reflejan este comportamiento: en Ridge, los coeficientes se reducen progresivamente pero permanecen activos, mientras que en LASSO varios coeficientes se anulan completamente a medida que aumenta la penalización. Esto evidencia que LASSO prioriza la parsimonia, conservando solo las variables más relevantes.

En conclusión, ambos modelos demostraron ser útiles y válidos para este dataset. La baja magnitud del MSE, la correcta selección de λ mediante validación cruzada, y la coherencia en los coeficientes obtenidos respaldan la calidad del ajuste. Ridge es ideal cuando se desea conservar toda la información disponible, mientras que LASSO es preferible cuando se busca

simplificar el modelo y destacar las variables más influyentes.

MODELOS DE CLASIFICACIÓN

Regresión Logística Multinomial

Implementación del modelo

1. Ajuste del modelo

```
#REGRESION LOGISTICA

x = vinos[["fixed acidity","volatile acidity","citric acid","residual sugar",
          "chlorides","free sulfur dioxide","total sulfur dioxide","density","pH","sulphates","alcohol"]]
y = [1 if i >= 6 else 0 for i in vinos["quality"]]

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.20, random_state=42)

# Corro la regresión logística
clf = LogisticRegression(random_state=0).fit(x_train, y_train)

# Miro los coeficientes que se obtuvieron
print(clf.coef_)

# Miro el intercept
print(clf.intercept_)
```

Ante la necesidad de clasificar el conjunto de datos utilizando la Regresión Logística Multinomial (LRM), determinamos las variables predictoras y ajustamos la variable respuesta (quality) implementando una condición que la separa en dos subconjuntos. La variable respuesta (y) fue transformada en binaria (siendo la condición primordial), asignando el valor 1 a los vinos con calidad mayor o igual a 6, y 0 a los de calidad inferior.

Posteriormente, se dividió el conjunto de datos en entrenamiento (80%) y prueba (20%) utilizando la función `train_test_split`, con una semilla fija (`random_state=42`) para garantizar la reproducibilidad de los resultados. El modelo fue ajustado sobre el conjunto de entrenamiento mediante la función `LogisticRegression().fit()`, que estima los coeficientes de cada variable predictora en función de su influencia sobre la probabilidad de pertenecer a la clase positiva (calidad ≥ 6).

Una vez entrenado el modelo, se imprimieron los coeficientes (`clf.coef_`) y el intercepto (`clf.intercept_`). Los coeficientes indican el peso de cada variable en la predicción: valores

positivos aumentan la probabilidad de que el vino sea clasificado como de buena calidad, mientras que valores negativos la disminuyen. El intercepto representa el sesgo del modelo, es decir, estima la probabilidad de que un caso pertenezca a la clase positiva.

2. Visualización de los Resultados e implementación de métricas

```
#Visualizacion de los resultados de la regresion logistica
X = vinos.drop("quality", axis=1)
y = [1 if i >= 6 else 0 for i in vinos["quality"]]

model = LogisticRegression(max_iter=1000)
model.fit(X, y)

plt.figure(figsize=(10,6))
plt.bar(X.columns, model.coef_[0])
plt.xticks(rotation=45)
plt.title("Importancia de cada variable en la regresión logística")
plt.ylabel("Coeficiente")
plt.grid(True)
plt.show()

# Calculo el accuracy en train y test
print(clf.score(x_train, y_train))
print(clf.score(x_test, y_test))
```

A partir de una mejor comprensión de la importancia relativa de cada variable predictora en el modelo, y evaluar el rendimiento en términos de precisión (accuracy) sobre los conjuntos de train y test. Se generó un gráfico de barras que determina el valor correspondiente de cada variable sobre la probabilidad de que un vino sea considerado dentro de la clase positiva (calidad ≥ 6).

Diagnóstico Visual

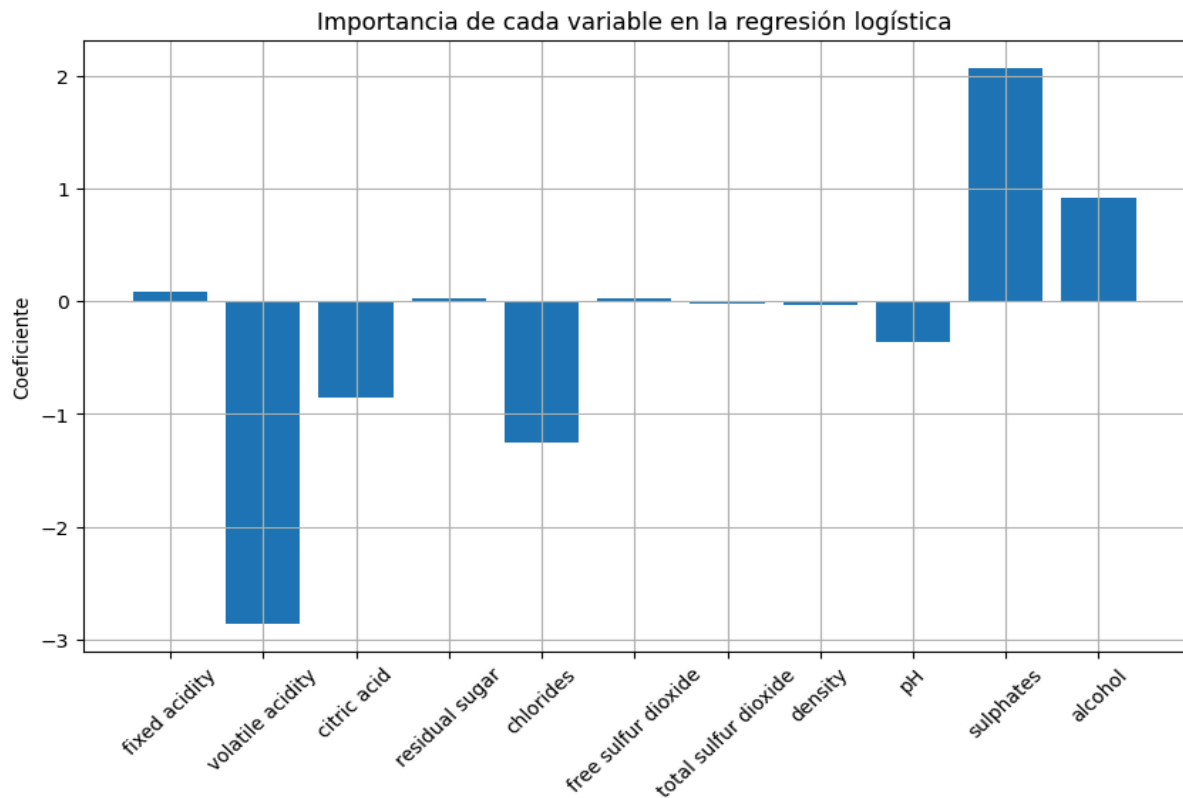


Figura 7: Gráfico de barras, acerca de la validación de la calidad del vino con respecto a las variables predictivas

Resultados del modelo

1. Ajuste del modelo

```
➡ [[-0.03921837 -2.76930072 -0.75024402 -0.01329634 -1.08064021 0.03011529  
    -0.02045166 -1.04734055 -1.68036261 1.82536159 0.92359608]]  
[-1.41637614]
```

2. Visualización de los resultados:

- 0.7522935779816514
- 0.7275

Evaluación del ajuste y validez del modelo

Al ejecutar el modelo de regresión logística podemos observar que la precisión obtenida fue de 0.74 en el conjunto de train y 0.72 en el conjunto de test, lo cual indica que el modelo

logra una clasificación aceptable y consistente. Esta cercanía entre ambos valores sugiere que el modelo no está sobreajustado y que puede predecir con buen nivel de acierto si un vino es de buena calidad utilizando únicamente sus propiedades.

Para interpretar la influencia de cada variable en la predicción, se entrenó un segundo modelo sobre el conjunto completo y se graficaron los coeficientes obtenidos. El gráfico de barras representa el peso de cada variable en la función de decisión del modelo: los coeficientes positivos indican que la variable aumenta la probabilidad de que el vino sea clasificado como de buena calidad, mientras que los coeficientes negativos la reducen. En este caso, las variables que contribuyen positivamente son free sulfur dioxide, sulphates (siendo esta la más influyente en sentido positivo), alcohol, residual sugar y fixed acidity. Por el contrario, variables como volatile acidity, pH, density, chlorides, citric acid y total sulfur dioxide presentan coeficientes negativos, lo que implica que están asociadas a vinos de menor calidad según el modelo.

Es importante destacar que los coeficientes pueden variar según el conjunto de datos utilizado para el entrenamiento. En el modelo solo con el conjunto de entrenamiento, fixed acidity presentaba un coeficiente negativo, mientras que en el modelo con todo el dataset aparece como positivo. Esta diferencia se debe a que los coeficientes reflejan relaciones que pueden cambiar levemente según la muestra utilizada. Para el análisis interpretativo, se considera más representativo el modelo entrenado con el conjunto completo, ya que incorpora toda la información disponible.

Además, el intercepto del modelo fue de -1.416 , lo que implica que, en ausencia de información (es decir, si todas las variables predictoras fueran cero), la probabilidad base de que un vino sea de buena calidad ≥ 6 es de aproximadamente **19.5%**. Este valor funciona como punto de partida del modelo, y luego los coeficientes ajustan esa probabilidad según los valores reales de cada muestra. En consecuencia, si la combinación de variables genera una probabilidad superior al umbral de 0.5, el vino será clasificado como de buena calidad; si no, se clasificará como de calidad inferior.

$$\text{Cálculo Aplicado: } \text{logit}(p) = -1.41637614 \Rightarrow p = \frac{1}{1 + e^{-(-1.41637614)}} \Rightarrow p = \frac{1}{1 + e^{1.41637614}}$$

$$\Rightarrow p = \frac{1}{1 + 4.122} \approx 0.195$$

En resumen, el modelo de regresión logística aplicado al dataset de vinos demuestra ser válido y útil, ya que ofrece una clasificación razonablemente precisa, permite interpretar el impacto de cada variable sobre la calidad percibida.

K vecinos más cercanos (KKN)

Implementación del Modelo

1. Normalización

Al momento de trabajar con K vecinos es necesaria la normalización de las variables para garantizar el cálculo de la distancia. Para ello se utilizó el comando `scaler = StandardScaler()`, el cual estandariza (media = 0 ; desvío estándar = 1) todas las variables predictoras. Mientras que `X_scaled = scaler.fit_transform(X)`, aplica la normalización a la matriz de variables predictoras.

Una vez normalizada dividimos el conjunto de datos en train (80%) y test (20%) utilizando la función `train_test_split`, con una semilla fija (`random_state=42`) para garantizar la reproducibilidad de los resultados.

```
#Normalizacion de las variables
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

2. Candidatos de K

Para alcanzar los candidatos de K determinamos un rango de valores posibles (`k_range = range(1,31)`). Posteriormente, almacenaremos en una lista (`cv_scores=[]`), que hasta el momento se encuentra vacía, los candidatos obtenidos.

Por consiguiente, para cada valor de K se entrena un modelo KNN y se aprecia su rendimiento a través de validación cruzada, permitiendo estimar el rendimiento de un modelo de forma más confiable. Al evaluar el modelo en múltiples particiones del conjunto de entrenamiento se reduce el riesgo.

Dicho modelo fue creado por `KNeighborsClassifier(n_neighbors=k)`, el cual posee k vecinos de

parámetro y al momento de entrenar el modelo lo realizamos con `scores = cross_val_score(knn, X_train, y_train, cv=5)`, que toma una partición de cinco de manera aleatoria.

Luego, calculamos el promedio de la predicción (`score.mean()`) para finalmente almacenarlo.

```
# Candidatos de k
k_range = range(1, 31)
cv_scores = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5)
    cv_scores.append(scores.mean())
```

3. Métricas de clasificación y elección del mejor K

Con el propósito de visualizar los valores logrados en el paso anterior se generó un gráfico que muestra cómo varía el accuracy promedio del modelo según el número de vecinos. Encontrándose en el eje horizontal los valores de K y en el eje vertical el accuracy promedio.

Además, a través de `best_k = k_range[np.argmax(cv_scores)]` se hizo un análisis automático de todos los accuracy promedio obteniendo el que lo maximiza. Este enfoque evita el sobreajuste que podría ocurrir con valores muy bajos y pérdida de sensibilidad con valores muy altos.

```
# Graficar accuracy vs k
plt.plot(k_range, cv_scores)
plt.xlabel("Número de vecinos (k)")
plt.ylabel("Accuracy promedio")
plt.title("Selección de k por validación cruzada")
plt.grid(True)
plt.show()

# Elegimos el mejor K
best_k = k_range[np.argmax(cv_scores)]
print("Mejor k:", best_k)
```

4. Entrenamiento del modelo Final

Al momento de entrenar el modelo final de KNN para luego generar predicciones (`y_pred=knn.final.predict(x_test)`) vamos a tener en cuenta el valor óptimo de K previamente

determinado, aplicándolo sobre datos nuevos, obteniendo una estimación realista de su rendimiento.

```
# Entrenamiento del modelo final
knn_final = KNeighborsClassifier(n_neighbors=best_k)
knn_final.fit(X_train, y_train)
y_pred = knn_final.predict(X_test)
```

5. Evaluación de KNN

Para concluir, utilizamos métricas de clasificación para la evaluación de KNN del modelo de entrenamiento final. En primera instancia, se generó la matriz de confusión que nos muestra la distribución de aciertos (aceptables) y errores (no aceptables) del modelo.

En base a `cm = confusion_matrix(y_test, y_pred)` calculamos la matriz comparando `y_test` con las predicciones del modelo `y_pred`. En cambio,

`cm_display=ConfusionMatrixDisplay(confusion_matrix = cm)` crea un objeto visual en base a los datos que después serán graficados.

Finalizando, con `print(classification_report(y_test, y_pred))` nos incluye un reporte completo de cuatro métricas (Precision ; Recall ; F1- score ; Accuracy)

```
#Evaluacion de KNN
cm = confusion_matrix(y_test, y_pred)
cm_display = ConfusionMatrixDisplay(confusion_matrix=cm)
cm_display.plot()
plt.title("Matriz de confusión sobre datos de test")
plt.show()

print("Reporte de clasificación:")
print(classification_report(y_test, y_pred))
```

Diagnóstico Visual

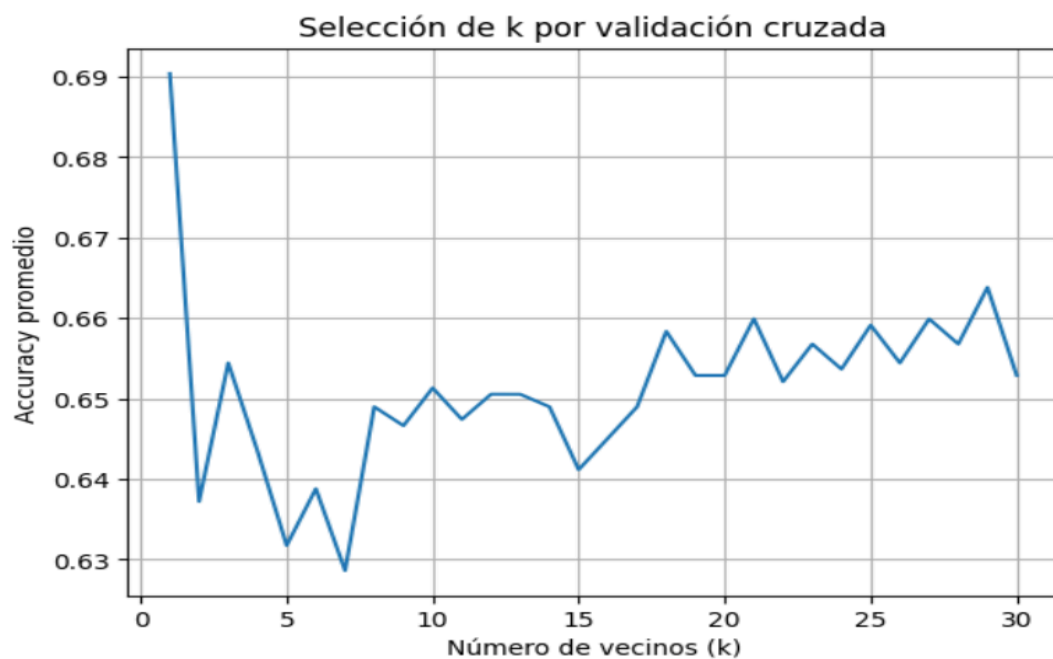
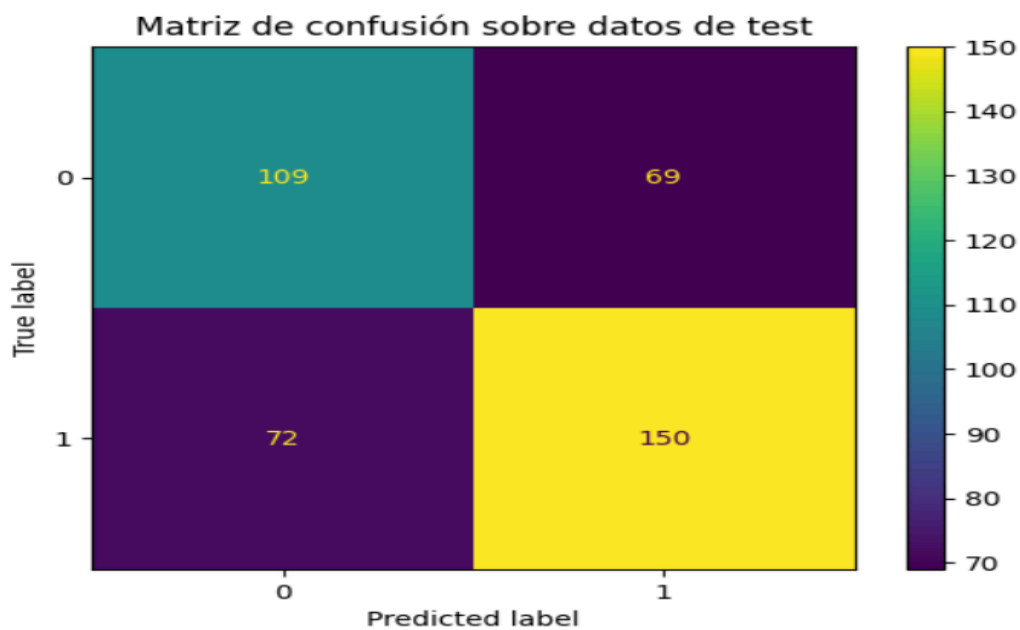


Figura 8: Gráfico de líneas, Calcula la precisión del modelo para cada valor de K

Figura 9: Matriz de confusión sobre acierto o error



Resultados del modelo

Mejor k : 1

Reporte de clasificación:				
	precision	recall	f1-score	support
0	0.60	0.61	0.61	178
1	0.68	0.68	0.68	222
accuracy			0.65	400
macro avg	0.64	0.64	0.64	400
weighted avg	0.65	0.65	0.65	400

Evaluación del ajuste y validez del modelo

El modelo de K Vecinos más Cercanos (KNN) se utiliza para clasificar observaciones en función de la similitud con otras muestras del conjunto de entrenamiento. Para ello, compara cada muestra nueva con sus *K vecinos más cercanos* y asigna la clase más frecuente entre ellos. Dado que se basa en distancias, se realizó una normalización previa de las variables mediante **StandardScaler**, asegurando que todas las características tengan la misma escala y no distorsionen el cálculo de cercanía.

Para evaluar el modelo se aplicó validación cruzada, que consiste en entrenar y probar el modelo varias veces con distintas divisiones del conjunto de entrenamiento. En cada iteración se calcula el accuracy, qué es el porcentaje de muestras correctamente clasificadas. Este valor sirve para medir qué tan bien está funcionando el modelo (+ alto = mejor clasificado).

En el gráfico de líneas se muestra cómo varía el accuracy promedio según el número de vecinos (K) utilizados por el modelo. El eje horizontal representa los valores de K evaluados (de 1 a 30), y el eje vertical muestra el accuracy promedio obtenido para cada uno. La curva permite visualizar qué valor de K ofrece el mejor rendimiento.

En este caso, el punto más alto del gráfico se encuentra en $K = 1$, lo que indica que ese valor fue el que logró mayor precisión promedio en la clasificación durante la validación cruzada.

Una vez seleccionado el mejor K, se entrenó el modelo final y se evaluó sobre el conjunto de pruebas. La matriz de confusión muestra que el modelo acertó 109 veces en la clase 0 (mala calidad) y 150 veces en la clase 1 (buena calidad), pero también cometió errores: 69 vinos de clase 0 fueron clasificados como clase 1, y 72 vinos de clase 1 fueron clasificados como clase 0.

El reporte de clasificación complementa la matriz de confusión, proporcionando métricas clave como precisión, recall, f1-score y support para cada clase. En este caso, el modelo obtuvo una precisión de que el 60% de los datos pertenecían a la clase 0 (vinos de mala calidad) y del 68% para la clase 1 (vinos de buena calidad).

Además, logró un recall del 61% para la clase 0 y del 68% para la clase 1, lo que indica que detectó correctamente el 61% de los vinos que realmente eran de mala calidad y el 68% de los que efectivamente eran de buena calidad.

El accuracy total fue de 0.65, lo que indica que el modelo clasificó correctamente el 65% de los vinos en el conjunto de prueba. Este valor se calcula como la suma de los aciertos (109 en clase 0 + 150 en clase 1 = 259) dividido por el total de muestras evaluadas (400), es decir: $\text{accuracy} = 259 / 400 = 0.6475 \approx 0.65$

Concluyendo, al combinar precisión y recall, el f1-score muestra que el modelo tiene un rendimiento más sólido en la clase 1, ya que ambas métricas son más altas en esa categoría. Esto sugiere que el modelo logra identificar con mayor eficacia los vinos de buena calidad, aunque presenta errores en ambas clases.

Bayes Ingenuo / QDA / LDA

Implementación del Modelo

1. Filtrado de Clases

A partir de evitar sesgos en la evaluación y resultados poco confiables, se hace un filtraje en el conjunto de datos para conservar únicamente aquellas variables que tienen al menos 30 observaciones

`(clases_validas = vinos["quality"].value_counts()[vinos["quality"].value_counts() >= 30].index)`. A través de esta línea contamos cuántas veces aparece cada valor de calidad, filtra sólo aquellas clases que tienen como mínimo 30 observaciones y extrae los valores de validación que cumplen esa condición.

En cambio, `vinos_filtrado = vinos[vinos["quality"].isin(clases_validas)]` guarda los valores filtrados en el código anterior.

```
#QDA , NB , LDA

# Filtrar clases con al menos 30 observaciones
clases_validas = vinos["quality"].value_counts()[vinos["quality"].value_counts() >= 30].index
vinos_filtrado = vinos[vinos["quality"].isin(clases_validas)]
```

2. Ajuste del modelo

Determinamos las variables predictoras y ajustamos la variable respuesta (quality) con los datos filtrados para luego dividirlo en train (80%) y test (20%) como lo veníamos haciendo a lo largo del informe.

Luego, definimos los modelos LDA, QDA y NB para futuro uso.

```
# Separar variables predictoras y objetivo
X = vinos_filtrado.drop("quality", axis=1)
y = vinos_filtrado["quality"]

# Dividir en entrenamiento y prueba
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# Definir modelos
lda = LinearDiscriminantAnalysis()
qda = QuadraticDiscriminantAnalysis()
nb = GaussianNB()

modelos = {
    "LDA": lda,
    "QDA": qda,
    "Naive Bayes": nb
}
```

3. Métricas y Evaluación

Sobre el conjunto, utilizando la calidad del vino (calidad mayor igual a 6). Se busca comparar su rendimiento en términos de precisión (accuracy) y métricas, para determinar cual posee mejor capacidad predictiva.

Para ello, en primera instancia entrenamos el modelo sobre el conjunto entrenamiento (`modelo.fit(x_train, y_train)`). En segunda instancia, generamos predicciones sobre el conjunto prueba (`y_pred = modelo.predict(x_test)`). Luego, calculamos la proporción de aciertos totales (`acc = accuracy_score(y_test, y_pred)`). Y para terminar con `print(classification_report(y_test, y_pred))` nos incluye un reporte completo de cuatro métricas (Precision ; Recall ; F1- score ; Accuracy)

```
#Entrenar, predecir y evaluar
resultados = {}

for nombre, modelo in modelos.items():
    modelo.fit(x_train, y_train)
    y_pred = modelo.predict(x_test)
    acc = accuracy_score(y_test, y_pred)
    resultados[nombre] = acc
    print(f"\n {nombre} - Accuracy: {acc:.3f}")
    print("Reporte de clasificación:")
    print(classification_report(y_test, y_pred))
```

4. Comparación visual entre NB/QDA/LDA

Con el propósito de visualizar los valores logrados en el paso anterior se generó un gráfico de barras que comparará los distintos modelos de clasificación LDA, QDA y NB para seleccionar posteriormente la opción acorde a lo que buscamos.

```
# Comparación visual
plt.figure(figsize=(8,5))
plt.bar(resultados.keys(), resultados.values())
plt.ylabel("Accuracy")
plt.title("Comparación de modelos: NB vs QDA vs LDA")
plt.ylim(0.2, 1.0)
plt.grid(True)
plt.show()
```

5. Matriz de confusión

Para concluir, se generaron la matrices de confusión que nos muestra el desempeño de los distintos modelos.

En primera medida `.predict(x_test)` lo utilizamos para generar predicciones de calidad.

En base a `confusion_matrix(y_test,y_pred)` calculamos la matriz comparando `y_test` con las predicciones del modelo `y_pred`. En cambio,

`ConfusionMatrixDisplay(cm_nb).plot` crea un objeto visual en base a los datos que después serán graficados.

```
#NB con la matriz confusion
y_pred_nb = nb.predict(x_test)
cm_nb = confusion_matrix(y_test, y_pred_nb)
ConfusionMatrixDisplay(cm_nb).plot()
plt.title("Matriz de confusión - Naive Bayes")
plt.show()
```

```
#QDA con la matriz confusion
y_pred_qda = qda.predict(x_test)
cm_qda = confusion_matrix(y_test, y_pred_qda)
ConfusionMatrixDisplay(cm_qda).plot()
plt.title("Matriz de confusión - QDA")
plt.show()

#LDA con la matriz confusion
y_pred_lda = lda.predict(x_test)
cm_lda = confusion_matrix(y_test, y_pred_lda)
ConfusionMatrixDisplay(cm_lda).plot()
plt.title("Matriz de confusión - LDA")
plt.show()
```

Diagnóstico Visual

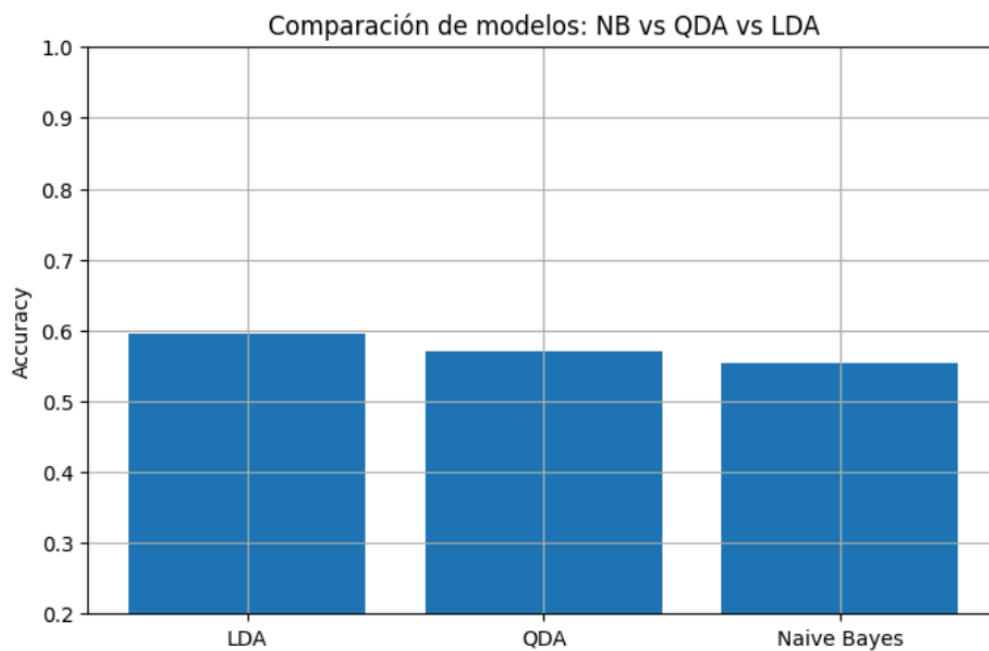


Figura 10: Gráfico de Barras que compara los modelos BN/QDA/LDA

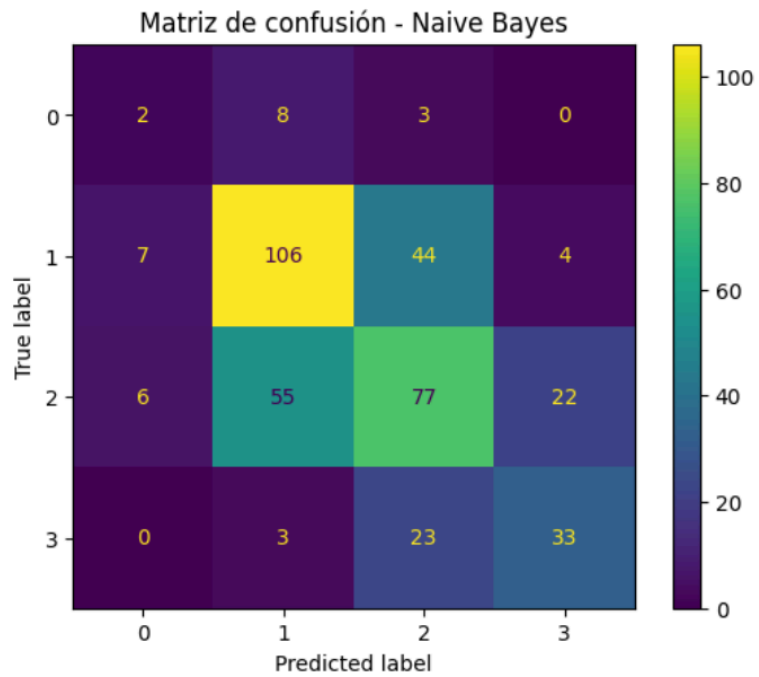


Figura 11: Matriz de confusión sobre los valores de Bayes Ingenuo

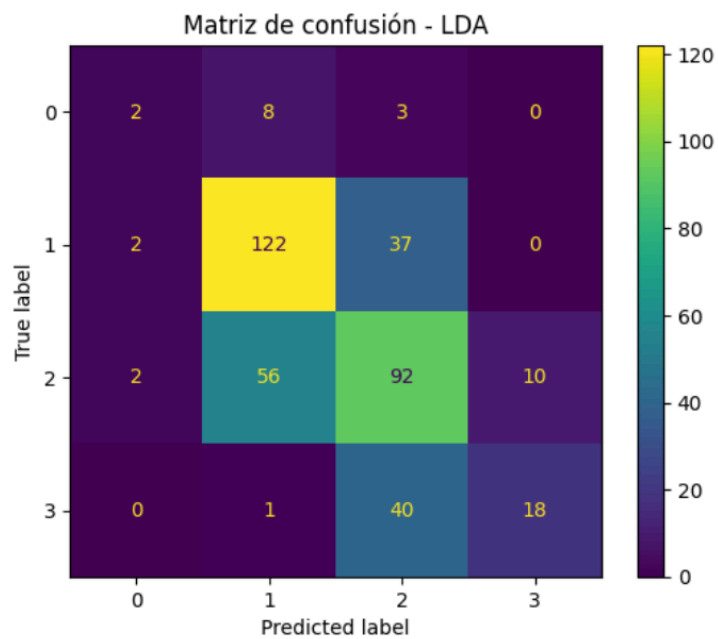


Figura 12: Matriz de Confusión sobre los valores de LDA

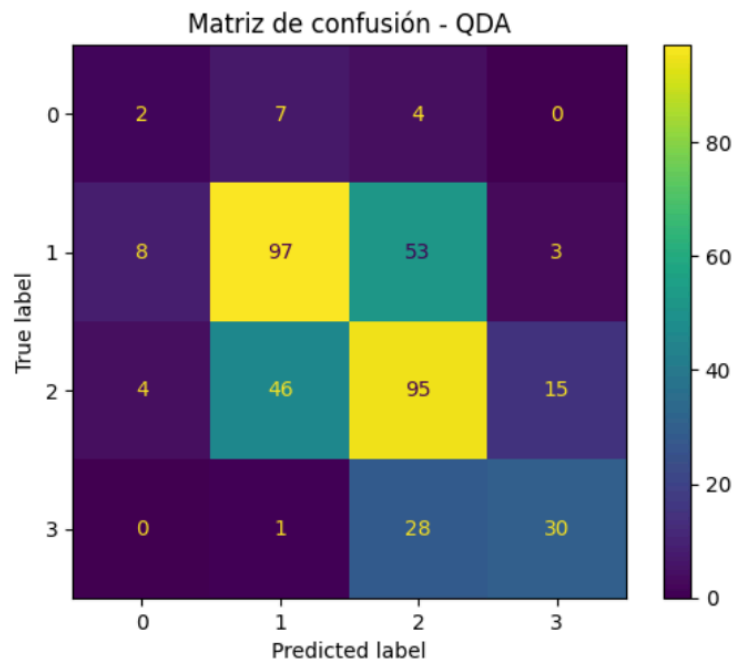


Figura 13: Matriz de confusión sobre los valores de QDA

Resultados del modelo

LDA - Accuracy: 0.595
 Reporte de clasificación:

	precision	recall	f1-score	support
4	0.33	0.15	0.21	13
5	0.65	0.76	0.70	161
6	0.53	0.57	0.55	160
7	0.64	0.31	0.41	59
accuracy			0.60	393
macro avg	0.54	0.45	0.47	393
weighted avg	0.59	0.60	0.58	393

QDA - Accuracy: 0.570
 Reporte de clasificación:

	precision	recall	f1-score	support
4	0.14	0.15	0.15	13
5	0.64	0.60	0.62	161
6	0.53	0.59	0.56	160
7	0.62	0.51	0.56	59
accuracy			0.57	393
macro avg	0.48	0.46	0.47	393
weighted avg	0.58	0.57	0.57	393

Naive Bayes - Accuracy: 0.555				
Reporte de clasificación:				
	precision	recall	f1-score	support
4	0.13	0.15	0.14	13
5	0.62	0.66	0.64	161
6	0.52	0.48	0.50	160
7	0.56	0.56	0.56	59
accuracy			0.55	393
macro avg	0.46	0.46	0.46	393
weighted avg	0.55	0.55	0.55	393

Figura 14: Reportes de Clasificación y valor del Accuracy en cada modelo

Evaluación del ajuste y validez del modelo

Se entrenaron tres modelos de clasificación sobre el dataset de vinos: Bayes Ingenuo (NB), Análisis Discriminante Cuadrático (QDA) y Análisis Discriminante Lineal (LDA). Para asegurar una distribución representativa, se filtraron las clases con al menos 30 observaciones, y se dividieron los datos en conjuntos de entrenamiento (80%) y prueba (20%). Cada modelo fue entrenado sobre el conjunto de entrenamiento y evaluado sobre el conjunto de prueba.

La métrica principal utilizada fue el accuracy, que representa el porcentaje de muestras correctamente clasificadas. Los resultados obtenidos fueron: LDA = 0.595, QDA = 0.570 y Naive Bayes = 0.555. Estos valores indican que LDA fue el modelo con mejor rendimiento general, seguido por QDA y luego Bayes Ingenuo. La diferencia entre ellos es moderada pero consistente, y se refleja también en los reportes de clasificación y las matrices de confusión.

Cada modelo generó un reporte de clasificación que incluye métricas clave como precisión, recall, f1-score y support para cada clase. La *precisión* indica cuántas veces el modelo acertó cuando predijo una clase; el *recall* muestra cuántos casos reales de esa clase fueron correctamente detectados; el *f1-score* combina ambas métricas en un promedio balanceado; y el *support* indica cuántas muestras reales pertenecen a cada clase.

Cuando hablamos de clases, hacemos referencia a la calidad como una variable categórica discreta, donde cada número representa una categoría distinta:

- Clase 4 → vinos de calidad baja
- Clase 5 → vinos de calidad media-baja
- Clase 6 → vinos de calidad media-alta
- Clase 7 → vinos de calidad alta

El modelo Naive Bayes mostró un desempeño limitado en la clase 4, con una precisión y un recall de apenas 0.18, lo que indica que el modelo no logra identificar correctamente los vinos de esa categoría. En contraste, en la clase 5 —la más representada— obtuvo un f1-score de 0.61, lo que refleja una clasificación aceptable. Las clases 6 y 7 presentaron resultados intermedios, con f1-scores de 0.47 y 0.58 respectivamente, lo que sugiere que el modelo tiene dificultades para distinguir correctamente entre las categorías más cercanas.

El modelo QDA presentó un comportamiento similar, con un f1-score de 0.20 en la clase 4, lo que confirma su baja capacidad para clasificar correctamente esa categoría. Sin embargo, logró un rendimiento más estable en las clases dominantes: f1-score de 0.59 en la clase 5, 0.50 en la clase 6 y 0.51 en la clase 7. Esto indica que QDA mejora ligeramente respecto a Naive Bayes en términos de balance entre precisión y recall en las clases frecuentes.

Por su parte, el modelo LDA fue el más equilibrado. Aunque también tuvo dificultades en la clase 4 (f1-score de 0.24), logró mejores resultados en las clases dominantes. En la clase 5 alcanzó un f1-score de 0.68, lo que representa el mejor desempeño entre todos los modelos. En la clase 6 obtuvo un f1-score de 0.54 y en la clase 7 de 0.42, lo que evidencia una mayor capacidad para distinguir entre categorías cercanas, aunque con margen de mejora en las clases menos representadas.

En conclusión, si bien ninguno de los modelos logra una clasificación precisa en la clase 4, LDA se destaca como el más robusto y equilibrado, con el mejor rendimiento en las clases más frecuentes y una mayor estabilidad general en sus métricas. Esto lo posiciona como el modelo más adecuado para abordar la clasificación multiclase en este dataset.

Las matrices de confusión permiten visualizar los aciertos y errores de cada modelo. Las celdas diagonales representan clasificaciones correctas, mientras que las celdas fuera de la diagonal indican errores. Los colores intensos muestran mayor cantidad de casos, facilitando la detección de patrones. En los tres modelos se observa que las clases más frecuentes (como 5 y 6) son mejor clasificadas, mientras que las clases menos representadas (como la 4) presentan mayor dispersión y errores. Esto sugiere que los modelos tienden a favorecer las

clases dominantes, lo cual es esperable en datasets desbalanceadas.

En conjunto, los tres modelos lograron una clasificación razonable, con LDA como el más efectivo en términos de precisión global y equilibrio entre clases. QDA y Bayes Ingenuo también mostraron buen desempeño en clases frecuentes, pero con mayor dificultad en clases minoritarias. Las métricas y visualizaciones respaldan la validez de los modelos como herramientas de clasificación, aunque podrían beneficiarse de técnicas adicionales como ajuste de parámetros, balanceo de clases o selección de variables.

CONCLUSIÓN

El análisis realizado sobre el dataset de vinos permitió contrastar diferentes enfoques de modelización, tanto desde la regresión como desde la clasificación. En el ámbito de la regresión, se observó que la regresión lineal múltiple, aunque estadísticamente significativa ($p\text{-value} < 0.001$), tiene un poder explicativo limitado y problemas de colinealidad (0.3561). En este sentido, los modelos penalizados demostraron ser más robustos: Ridge resultó adecuado para mantener todas las variables estabilizando los coeficientes (MSE 0.3425748), mientras que LASSO permitió simplificar el modelo seleccionando solo los predictores más influyentes, aunque con un desempeño similar (MSE 0.3442939).

En cuanto a los modelos de clasificación, la regresión logística multinomial mostró un rendimiento sólido y equilibrado con promedio de precisión del 0.74 en el conjunto de train y 0.72 en el conjunto de test, obteniendo una buena capacidad predictiva e interpretabilidad, superando a KNN, Bayes ingenuo, LDA y QDA en precisión y consistencia. KNN logró un nivel aceptable de acierto, pero su sensibilidad al ruido y a la escala lo vuelve menos confiable. LDA y QDA estuvieron limitados por sus supuestos estadísticos, mientras que Bayes ingenuo evidenció un bajo desempeño al no capturar adecuadamente las dependencias entre variables.

De este modo, concluimos que Ridge es el modelo más adecuado para la regresión, mientras que la regresión logística multinomial es el mejor clasificador para este dataset. Esta combinación ofrece la mejor relación entre robustez, estabilidad y capacidad predictiva, constituyendo una base sólida para el análisis de la calidad del vino.